Part 2: Integration Testing Report

StreetRace Manager

Amay Sharma

March 23, 2026

1 Call Graph

Figure 1: Hand-drawn call graph for StreetRace Manager.

2 Module Descriptions

The integration implementation contains 8 modules: 6 mandatory + 2 custom modules.

1) Registration Module (Mandatory)

- Responsibility: register crew members and maintain canonical identity records.
- Core entities: `Role`, `CrewMember`, `RegistrationModule`.
- Key operations: `register_member`, `get_member`, `get_member_by_name`, `remove_member`, `count`.
- Validation: rejects empty/whitespace names; guarantees incrementing member IDs.

2) Crew Management Module (Mandatory)

- Responsibility: role assignment, skill updates, availability management, and team creation.
- Depends on: `RegistrationModule`.
- Key operations: `assign_role`, `update_skill_level`, `set_availability`, `get_available_by_role`, `get_best_drive`
- Validation: raises `MemberNotFoundError` for unknown members and checks skill range (1–10).

3) Inventory Module (Mandatory)

- Responsibility: manage funds, cars, parts, and tools.
- Core entities: `Car`, `SparePart`, `Tool`, `CarCondition`.
- Key operations: `add_funds`/`deduct_funds`, `add_car`, `get_race_ready_cars`, `repair_car`, `use_parts`.
- Validation: positive amount checks and `InsufficientFundsError` for balance underflow.

4) Race Management Module (Mandatory)

- Responsibility: create races, validate entries, and transition race lifecycle.
- Depends on: `CrewManagementModule`, `InventoryModule`.
- Core entities: `Race`, `RaceEntry`, `RaceType`, `RaceStatus`.
- Key operations: `create_race`, `enter_race`, `start_race`, `cancel_race`, `auto_select_best_entry`.
- Validation: only available drivers with role DRIVER and race-ready cars can enter.

5) Results Module (Mandatory)

- Responsibility: finalize races, distribute prizes, and update cumulative driver stats.
- Depends on: `RaceManagementModule`, `InventoryModule`.
- Core entities: `RaceOutcome`, `RaceResult`, `DriverStats`.
- Key operations: `record_race_results`, `get_driver_stats`, `get_rankings`.
- Business logic: winner 50%, runner-up 30%, third 15%, remaining 5% shared.

6) Mission Planning Module (Mandatory)

- Responsibility: mission lifecycle planning and role/car requirement enforcement.
- Depends on: `CrewManagementModule`, `InventoryModule`.
- Core entities: `Mission`, `MissionType`, `MissionStatus`, `MissionAssignment`.
- Key operations: `create_mission`, `assign_crew_to_mission`, `assign_car_to_mission`, `start_mission`, `complete_mission`, `cancel_mission`, `auto_assign_mission`.
- Validation: mission cannot start until all required roles are assigned; driver-role missions require a car.

7) Leaderboard Module (Custom Module 1)

- Responsibility: compute ranked performance views and achievements.
- Depends on: `ResultsModule`.
- Key operations: `sync_from_results`, `get_leaderboard`, `record_mission_completion`, `get_achievements`.
- Supports ranking dimensions: race wins, earnings, podiums, mission completions, and overall weighted score.

8) Notifications Module (Custom Module 2)

- Responsibility: event notifications, targeted/broadcast delivery, read tracking.
- Core entities: `Notification`, `NotificationType`, `NotificationCategory`, `NotificationPriority`.
- Key operations: `create_notification`, `notify_race_starting`, `notify_race_result`, `notify_mission_availabl` `notify_low_funds`.
- Integration role: decouples event generation from UI/logging sinks via handler callbacks.

3 Integration Test Design

Test suite source: `dass-assignment-2/2024101095/integration/tests/test_*.py`

Scenario Tested	Modules Involved	Expected Result	Actual Result	Errors / Logical Issues Found and Fixed
Complete race workflow (register, create race, enter, start, record)	Registration, Inventory, Race Mgmt, Results, Notifications	Winner recorded; prize added; resources released	Passed	Earlier logic issues in related white-box flow (trade/prize accounting style bugs) were corrected before final integration run.
Complete mission workflow (assign crew+car, start, complete)	Crew Mgmt, Inventory, Mission Planning, Notifications	Mission status transitions to COMPLETED and reward credited	Passed	Requirement checks (role+car) validated through `start_mission` guards.
Resource conflict: same driver/car in mission and race	Mission Planning, Race Mgmt	Race entry should be rejected with `InvalidEntryError`	Passed	Prevents double booking and stale availability state.
Leaderboard reflects winners after race outcomes	Results, Leaderboard	Winning driver appears at rank 1 for race-wins board	Passed	Sync path `sync_from_results()` verified.
Notification emission during race + mission lifecycle	Notifications, Race Mgmt, Results, Mission Planning	Expected race-category notifications are generated and retrievable	Passed	Handler registration/dispatch and category tagging validated.
Mission failure should not add funds	Mission Planning, Inventory	FAILED mission keeps balance unchanged	Passed	Confirms failure branch does not leak reward credits.

Scenario Tested	Modules Involved	Expected Result	Actual Result	Errors / Logical Issues Found and Fixed
Multiple races accumulate long-term stats	Registration, Race Mgmt, Results	`races_entered`, `races_won`, `podium_finishes` increment cumulatively	Passed	Prevents stat reset/regression across repeated races.
Role availability + missing-role checks before mission start	Crew Mgmt, Mission Planning	Missing roles are reported and mission start blocked	Passed	Guards against partial crew assignment bugs.
Inventory constraints (fund underflow, repair cost, part quantity)	Inventory	Errors raised for invalid deductions/consumption	Passed	Economic invariants preserved.
Race entry validation for non-driver, unavailable driver, bad car state	Race Mgmt, Crew Mgmt, Inventory	Entry rejected with explicit errors	Passed	Avoids inconsistent race participants and invalid car condition usage.

3.1 Complete Test Case Matrix (All Integration Tests)

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Members added through registration appear in crew queries.	crew_management._registration.register_member("Alice", Role.DRIVER); crew_management.get_members_by_role(Role.DRIVER); len(drivers)	Members added through registration appear in crew queries.; assert len(drivers) == 1; assert drivers[0].name == "Alice"	PASSED (assertions satisfied)	tests/test_crew_management.TestCrewManagementI test_registered_member
Members with different roles appear in the right buckets.	crew_management._registration.register_member("Alice", Role.DRIVER); crew_management._registration.register_member("Bob", Role.MECHANIC);...	Members with different roles appear in the right buckets.; assert len(crew_management.get_members_by_role(Role.DRIVER)) == 1; assert len(crew_management.get_...	PASSED (assertions satisfied)	tests/test_crew_management.TestCrewManagementI test_multiple_roles_corr
A member registered without role can be given one later.	crew_management._registration.register_member("Dave"); crew_management.assign_role(member.member_id, Role.STRATEGIST)	A member registered without role can be given one later.; assert member.role is None; assert updated.role == Role.STRATEGIST	PASSED (assertions satisfied)	tests/test_crew_management.TestCrewManagementI test_assign_role_after_re
Assigning a role to a non-existent member raises MemberNotFoundError.	crew_management.assign_role(999, Role.DRIVER)	Assigning a role to a non-existent member raises MemberNotFoundError.; Expect exception via pytest.raises(MemberNotFoundError)	PASSED (assertions satisfied)	tests/test_crew_management.TestCrewManagementI test_assign_role_to_unkn

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Skill level can be updated within the 1-10 range.	crew_management._registration.register_member("Eva", Role.DRIVER); crew_management.update_skill_level(member.member_id, 8)	Skill level can be updated within the 1-10 range.; assert member.skill_level == 8	PASSED (assertions satisfied)	tests/test_crew_management.TestCrewManagementI test_skill_level_update
Skill level outside 1-10 raises ValueError.	crew_management._registration.register_member("Frank", Role.MECHANIC); crew_management.update_skill_level(member.member_id, 11); crew_man...	Skill level outside 1-10 raises ValueError.; Expect exception via pytest.raises(ValueError); Expect exception via pytest.raises(ValueError)	PASSED (assertions satisfied)	tests/test_crew_management.TestCrewManagementI test_skill_level_out_of_ra
Unavailable members are excluded from available-role queries.	crew_management._registration.register_member("G", Role.DRIVER); crew_management._registration.register_member("H", Role.DRIVER); crew_ma...	Unavailable members are excluded from available-role queries.; assert len(available) == 1; assert available[0].member_id == m2.member_id	PASSED (assertions satisfied)	tests/test_crew_management.TestCrewManagementI test_availability_filterin
has_available_role reflects current availability correctly.	crew_management._registration.register_member("I", Role.DRIVER); crew_management.has_available_role(Role.DRIVER)	has_available_role reflects current availability correctly.; assert crew_management.has_available_role(Role.DRIVER) is False; assert crew_management.has_avai...	PASSED (assertions satisfied)	tests/test_crew_management.TestCrewManagementI test_has_available_role
set_availability updates the member's availability flag.	crew_management._registration.register_member("J", Role.PIT_CREW); crew_management.set_availability(member.member_id, False); crew_manage...	set_availability updates the member's availability flag.; assert member.is_available is False; assert member.is_available is True	PASSED (assertions satisfied)	tests/test_crew_management.TestCrewManagementI test_set_availability_via
get_best_driver returns the available driver with the most skill.	reg.register_member("Low Skill", Role.DRIVER); reg.register_member("High Skill", Role.DRIVER); crew_management.update_skill_level(m1.memb...	get_best_driver returns the available driver with the most skill.; assert best is not None; assert best.member_id == m2.member_id	PASSED (assertions satisfied)	tests/test_crew_management.TestCrewManagementI test_get_best_driver_pic
create_team stores the team and retrieve it back.	reg.register_member("K", Role.DRIVER); reg.register_member("L", Role.MECHANIC); crew_management.create_team("Alpha", [m1.member_id, m2.me...	create_team stores the team and retrieve it back.; assert len(team) == 2; assert fetched is not None; assert m1.member_id in member_ids	PASSED (assertions satisfied)	tests/test_crew_management.TestCrewManagementI test_create_team
Full lifecycle: register drivers → add cars → create race → enter entries → start → record results → verify stats and prize money.	reg.register_member("Lightning", Role.DRIVER); reg.register_member("Thunder", Role.DRIVER); inv.add_car("Speed Demon", "Sports", 450)	Full lifecycle: register drivers → add cars → create race → enter entries → start → record results → verify stats and prize money.; assert outcome.results[0]...	PASSED (assertions satisfied)	tests/test_full_system.py::TestFullSystemWorkflo test_complete_race_worl

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Full lifecycle: register crew → create mission → assign crew + car → start mission → complete → verify balance and status.	crew._registration.register_member("Rescue Driver", Role.DRIVER); crew._registration.register_member("Rescue Mech", Role.MECHANIC); inv.a...	Full lifecycle: register crew → create mission → assign crew + car → start mission → complete → verify balance and status.; assert mission_mod.can_start_miss...	PASSED (assertions satisfied)	tests/test_full_system.py::TestFullSystemWorkflo test_complete_mission_v
A driver assigned to a mission cannot simultaneously enter a race.	crew._registration.register_member("Busy Driver", Role.DRIVER); inv.add_car("Only Car", "Sports", 400); mission_mod.create_mission("Job",...	A driver assigned to a mission cannot simultaneously enter a race.; Expect exception via pytest.raises(InvalidEntryError)	PASSED (assertions satisfied)	tests/test_full_system.py::TestFullSystemWorkflo test_resource_conflict_dr
After a race, the leaderboard should rank the winner at position 1.	reg.register_member("Champ", Role.DRIVER); reg.register_member("Runner", Role.DRIVER); inv.add_car("Rocket", "Sports", 500)	After a race, the leaderboard should rank the winner at position 1.; assert board[0].member.member_id == d1.member_id	PASSED (assertions satisfied)	tests/test_full_system.py::TestFullSystemWorkflo test_leaderboard_reflect
Race and mission events generate the expected notifications.	reg.register_member("Notif Driver", Role.DRIVER); inv.add_car("Fast Car", "Model", 400); race_mgmt.create_race("Notif Race", RaceType.CIR...	Race and mission events generate the expected notifications.; assert NotificationCategory.RACE in categories	PASSED (assertions satisfied)	tests/test_full_system.py::TestFullSystemWorkflo test_notifications_sent_d
A failed mission should not increase the inventory balance.	crew._registration.register_member("Risky Driver", Role.DRIVER); inv.add_car("Clunker", "Unknown", 100); inv.get_balance()	A failed mission should not increase the inventory balance.; assert inv.get_balance() == before; assert m.status == MissionStatus.FAILED	PASSED (assertions satisfied)	tests/test_full_system.py::TestFullSystemWorkflo test_mission_failure_doe
Stats accumulate correctly across multiple race wins.	reg.register_member("Veteran", Role.DRIVER); range(3); results.get_driver_stats(driver.member_id)	Stats accumulate correctly across multiple race wins.; assert stats.races_entered == 3; assert stats.races_won == 3; assert stats.podium_finishes == 3	PASSED (assertions satisfied)	tests/test_full_system.py::TestFullSystemWorkflo test_multiple_races_accu
Starting balance is set at creation time.	inventory.get_balance()	Starting balance is set at creation time.; assert inventory.get_balance() == 10_000.0	PASSED (assertions satisfied)	tests/test_inventory.py::TestInventoryModule::test_initial_balance
Adding funds increases the balance.	inventory.add_funds(5_000.0); inventory.get_balance()	Adding funds increases the balance.; assert inventory.get_balance() == 15_000.0	PASSED (assertions satisfied)	tests/test_inventory.py::TestInventoryModule::test_add_funds
Deducting funds decreases the balance.	inventory.deduct_funds(3_000.0); inventory.get_balance()	Deducting funds decreases the balance.; assert inventory.get_balance() == 7_000.0	PASSED (assertions satisfied)	tests/test_inventory.py::TestInventoryModule::test_deduct_funds
Deducting more than available raises InsufficientFundsError.	inventory.deduct_funds(999_999.0)	Deducting more than available raises InsufficientFundsError.; Expect exception via pytest.raises(InsufficientFundsError)	PASSED (assertions satisfied)	tests/test_inventory.py::TestInventoryModule::test_deduct_more_than_

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Adding a non-positive amount raises ValueError.	inventory.add_funds(-100.0)	Adding a non-positive amount raises ValueError.; Expect exception via pytest.raises(ValueError)	PASSED (assertions satisfied)	tests/ test_inventory.py:: TestInventoryModule:: test_add_negative_amou
A car added to inventory can be retrieved by ID.	inventory.add_car("Eclipse", "Mitsubishi", 280); inventory.get_car(car.car_id)	A car added to inventory can be retrieved by ID.; assert retrieved is not None; assert retrieved.name == "Eclipse"; assert retrieved.model == "Mitsubishi"	PASSED (assertions satisfied)	tests/ test_inventory.py:: TestInventoryModule:: test_add_car_and_retriev
Cars are added in GOOD condition by default.	inventory.add_car("Civic", "Honda", 200)	Cars are added in GOOD condition by default.; assert car.condition == CarCondition.GOOD	PASSED (assertions satisfied)	tests/ test_inventory.py:: TestInventoryModule:: test_default_car_conditi
Car condition can be updated.	inventory.add_car("Supra", "Toyota", 380); inventory.set_car_condition(car.car_id, CarCondition.DAMAGED)	Car condition can be updated.; assert car.condition == CarCondition.DAMAGED	PASSED (assertions satisfied)	tests/ test_inventory.py:: TestInventoryModule:: test_set_car_condition
get_available_cars excludes unavailable or totaled cars.	inventory.add_car("Charger", "Dodge", 450); inventory.add_car("Challenger", "Dodge", 420); inventory.get_available_cars()	get_available_cars excludes unavailable or totaled cars.; assert c1.car_id not in ids; assert c2.car_id in ids	PASSED (assertions satisfied)	tests/ test_inventory.py:: TestInventoryModule:: test_get_available_cars_e
get_race_ready_cars excludes damaged cars.	inventory.add_car("GTR", "Nissan", 550); inventory.add_car("Beater", "Generic", 100); inventory.set_car_condition(bad_car.car_id, CarCond...	get_race_ready_cars excludes damaged cars.; assert good_car.car_id in ids; assert bad_car.car_id not in ids	PASSED (assertions satisfied)	tests/ test_inventory.py:: TestInventoryModule:: test_get_race_ready_cars
repair_car restores a damaged car to GOOD condition.	inventory.add_car("Old Banger", "Unknown", 120); inventory.set_car_condition(car.car_id, CarCondition.DAMAGED); inventory.repair_car(car....	repair_car restores a damaged car to GOOD condition.; assert car.condition == CarCondition.GOOD	PASSED (assertions satisfied)	tests/ test_inventory.py:: TestInventoryModule:: test_repair_car_restores_
repair_car deducts the repair cost from the balance.	inventory.add_car("Car", "Model", 200); inventory.set_car_condition(car.car_id, CarCondition.DAMAGED); inventory.get_balance()	repair_car deducts the repair cost from the balance.; assert inventory.get_balance() == before - 1_000.0	PASSED (assertions satisfied)	tests/ test_inventory.py:: TestInventoryModule:: test_repair_costs_money
Parts can be added and quantities decremented.	inventory.add_part("Engine Filter", 10, 25.0); inventory.use_parts(part.part_id, 3); inventory.get_part(part.part_id)	Parts can be added and quantities decremented.; assert updated.quantity == 7	PASSED (assertions satisfied)	tests/ test_inventory.py:: TestInventoryModule:: test_add_part_and_use
Using more parts than in stock raises ValueError.	inventory.add_part("Brake Pad", 2, 40.0); inventory.use_parts(part.part_id, 5)	Using more parts than in stock raises ValueError.; Expect exception via pytest.raises(ValueError)	PASSED (assertions satisfied)	tests/ test_inventory.py:: TestInventoryModule:: test_use_more_parts_tha
Tools can be added and listed.	inventory.add_tool("Impact Wrench"); inventory.add_tool("Jack Stand"); inventory.get_available_tools()	Tools can be added and listed.; assert len(tools) == 2	PASSED (assertions satisfied)	tests/ test_inventory.py:: TestInventoryModule:: test_add_tool_and_list

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
get_inventory_summary returns a dict with expected keys.	inventory.add_car("Car", "Model", 200); inventory.get_inventory_summary()	get_inventory_summary returns a dict with expected keys.; assert "cash_balance" in summary; assert "total_cars" in summary; assert summary["total_cars"] == 1	PASSED (assertions satisfied)	tests/test_inventory.py::TestInventoryModule::test_inventory_summary
Fresh leaderboard has no entries.	leaderboard.get_leaderboard(LeaderboardType.RACE_WINS)	Fresh leaderboard has no entries.; assert board == []	PASSED (assertions satisfied)	tests/test_leaderboard.py::TestLeaderboardIntegra test_leaderboard_starts_
After a race is recorded, leaderboard reflects the results.	_run_race_and_record(results); leaderboard.sync_from_results(); leaderboard.get_leaderboard(LeaderboardType.RACE_WINS, limit=5)	After a race is recorded, leaderboard reflects the results.; assert len(board) ¿= 1; assert board[0].member.member_id == winner.member_id	PASSED (assertions satisfied)	tests/test_leaderboard.py::TestLeaderboardIntegra test_leaderboard_popula
Total earnings leaderboard shows the correct amounts.	_run_race_and_record(results, prize_pool=10_000.0); leaderboard.sync_from_results(); leaderboard.get_leaderboard(LeaderboardType.TOTAL_EA...	Total earnings leaderboard shows the correct amounts.; assert board[0].member.member_id == winner.member_id; assert board[0].score == 5_000.0	PASSED (assertions satisfied)	tests/test_leaderboard.py::TestLeaderboardIntegra test_earnings_leaderboa
OVERALL leaderboard uses the weighted formula.	_run_race_and_record(results); leaderboard.sync_from_results(); leaderboard.get_leaderboard(LeaderboardType.OVERALL)	OVERALL leaderboard uses the weighted formula.; assert winner_entry is not None; assert winner_entry.rank == 1	PASSED (assertions satisfied)	tests/test_leaderboard.py::TestLeaderboardIntegra test_overall_leaderboard
Mission completions can be tracked independently of race results.	leaderboard.record_mission_completion(member_id=42, success=True, earnings=500.0); leaderboard._performance.get(42)	Mission completions can be tracked independently of race results.; assert perf is not None; assert perf.missions_completed == 1; assert perf.total_earnings =...	PASSED (assertions satisfied)	tests/test_leaderboard.py::TestLeaderboardIntegra test_mission_completion
Failed missions reduce the overall score.	leaderboard.record_mission_completion(member_id=7, success=True); leaderboard.record_mission_completion(member_id=7, success=False)	Failed missions reduce the overall score.; assert score_after_fail ¡ score_after_win	PASSED (assertions satisfied)	tests/test_leaderboard.py::TestLeaderboardIntegra test_mission_failures_re
A driver who wins their first race earns 'first_win' achievement.	_run_race_and_record(results); leaderboard.sync_from_results(); leaderboard._check_achievements(winner.member_id)	A driver who wins their first race earns 'first_win' achievement.; assert "first_win" in achieved_ids	PASSED (assertions satisfied)	tests/test_leaderboard.py::TestLeaderboardIntegra test_first_win_achieveme
get_member_rank returns correct rank for a driver.	_run_race_and_record(results); leaderboard.sync_from_results(); leaderboard._check_achievements(winner.member_id)	get_member_rank returns correct rank for a driver.; assert rank == 1	PASSED (assertions satisfied)	tests/test_leaderboard.py::TestLeaderboardIntegra test_get_member_rank
format_leaderboard returns a non-empty readable string.	_run_race_and_record(results); leaderboard.sync_from_results(); leaderboard.format_leaderboard(LeaderboardType.RACE_WINS)	format_leaderboard returns a non-empty readable string.; assert isinstance(output, str); assert "LEADERBOARD" in output	PASSED (assertions satisfied)	tests/test_leaderboard.py::TestLeaderboardIntegra test_format_leaderboard

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
DELIVERY missions need exactly one DRIVER.	crew._registration.register_member("Driver", Role.DRIVER); mission_planning.can_start_mission(MissionType.DELIVERY)	DELIVERY missions need exactly one DRIVER.; assert mission_planning.can_start_mission(MissionType.DELIVERY) is False; assert mission_planning.can_start_missi...	PASSED (assertions satisfied)	tests/ test_mission_planning.p TestMissionPlanningInt test_delivery_requires_d
RESCUE missions need both DRIVER and MECHANIC.	set(mission_planning.get_missing_roles(MissionType.RESCUE)); crew._registration.register_member("Driver", Role.DRIVER); crew._registratio...	RESCUE missions need both DRIVER and MECHANIC.; assert Role.DRIVER in missing; assert Role.MECHANIC in missing; assert Role.DRIVER not in missing	PASSED (assertions satisfied)	tests/ test_mission_planning.p TestMissionPlanningInt test_rescue_requires_dri
A crew member assigned to a mission becomes unavailable.	mission_planning._crew._registration.register_member("D", Role.DRIVER); mission_planning.create_mission("Job", MissionType.DELIVERY, 1_00...	A crew member assigned to a mission becomes unavailable.; assert driver.is_available is True; assert driver.is_available is False	PASSED (assertions satisfied)	tests/ test_mission_planning.p TestMissionPlanningInt test_assigning_crew_mar
A mission with a DRIVER role cannot start without an assigned car.	mission_planning._crew._registration.register_member("D", Role.DRIVER); mission_planning.create_mission("Careless", MissionType.DELIVERY,...	A mission with a DRIVER role cannot start without an assigned car.; Expect exception via pytest.raises(MissionError, match="car")	PASSED (assertions satisfied)	tests/ test_mission_planning.p TestMissionPlanningInt test_mission_needs_car_
Completing a mission frees crew member and car.	crew._registration.register_member("Driver", Role.DRIVER); inv.add_car("Van", "Ford", 200); mission_planning.create_mission("Delivery", M...	Completing a mission frees crew member and car.; assert driver.is_available is False; assert car.is_available is False; assert driver.is_available is True	PASSED (assertions satisfied)	tests/ test_mission_planning.p TestMissionPlanningInt test_complete_mission_r
Completing a mission successfully adds reward to inventory.	inv.get_balance(); mission_planning._crew._registration.register_member("D", Role.DRIVER); inv.add_car("Truck", "Chevy", 250)	Completing a mission successfully adds reward to inventory.; assert inv.get_balance() == before + 3_000.0	PASSED (assertions satisfied)	tests/ test_mission_planning.p TestMissionPlanningInt test_successful_mission_
A failed mission does not credit any reward.	inv.get_balance(); mission_planning._crew._registration.register_member("D", Role.DRIVER); inv.add_car("Clunker", "Unknown", 100)	A failed mission does not credit any reward.; assert inv.get_balance() == before	PASSED (assertions satisfied)	tests/ test_mission_planning.p TestMissionPlanningInt test_failed_mission_no_r
A car that is damaged during a mission gets DAMAGED condition.	mission_planning._crew._registration.register_member("D", Role.DRIVER); inv.add_car("Risky Car", "Coupe", 300); mission_planning.create_m...	A car that is damaged during a mission gets DAMAGED condition.; assert car.condition == CarCondition.DAMAGED	PASSED (assertions satisfied)	tests/ test_mission_planning.p TestMissionPlanningInt test_car_damaged_durin

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Cancelling a mission in ASSIGNED state frees crew and car.	crew._registration.register_member("D", Role.DRIVER); inv.add_car("Car", "Model", 200); mission_planning.create_mission("Cancel This", Mi...	Cancelling a mission in ASSIGNED state frees crew and car.; assert driver.is_available is True; assert car.is_available is True; assert mission.status == Mis...	PASSED (assertions satisfied)	tests/ test_mission_planning.p TestMissionPlanningInt test_cancel_mission_rele
auto_assign_mission picks highest-skill driver.	crew._registration.register_member("Low", Role.DRIVER); crew._registration.register_member("High", Role.DRIVER); crew.update_skill_level(...	auto_assign_mission picks highest-skill driver.; assert high.is_available is False; assert low.is_available is True	PASSED (assertions satisfied)	tests/ test_mission_planning.p TestMissionPlanningInt test_auto_assign_selects
A notification can be created with title, message, type, category.	notifications.create_notification("Test Alert", "Something happened", NotificationType.INFO, NotificationCategory.SYSTEM)	A notification can be created with title, message, type, category.; assert notif.notification_id == 1; assert notif.title == "Test Alert"; assert notif.read ...	PASSED (assertions satisfied)	tests/ test_notifications.py:: TestNotificationsModul test_create_basic_notific
Notification IDs increment sequentially.	notifications.create_notification("N1", "msg", NotificationType.INFO, NotificationCategory.SYSTEM); notifications.create_notification(...	Notification IDs increment sequentially.; assert n2.notification_id == n1.notification_id + 1	PASSED (assertions satisfied)	tests/ test_notifications.py:: TestNotificationsModul test_notification_ids_aut
mark_as_read flips the read flag and records timestamp.	notifications.create_notification("Unread", "msg", NotificationType.INFO, NotificationCategory.SYSTEM); notifications.mark_as_read(noti...	mark_as_read flips the read flag and records timestamp.; assert notif.read is False; assert notif.read is True; assert notif.read_at is not None	PASSED (assertions satisfied)	tests/ test_notifications.py:: TestNotificationsModul test_mark_as_read
Registered handlers are called when a notification is created.	notifications.register_handler(handler); notifications.create_notification("CB Test", "message", NotificationType.INFO, NotificationCate...	Registered handlers are called when a notification is created.; assert len(received) == 1	PASSED (assertions satisfied)	tests/ test_notifications.py:: TestNotificationsModul test_handler_callback_is
Unregistered handlers are not called.	notifications.register_handler(handler); notifications.unregister_handler(handler); notifications.create_notification("Silent", "msg", N...	Unregistered handlers are not called.; assert len(received) == 0	PASSED (assertions satisfied)	tests/ test_notifications.py:: TestNotificationsModul test_unregister_handler
A broadcast notification (no target_member_id) reaches all members.	notifications.create_notification("Broadcast", "Everyone sees this", NotificationType.INFO, NotificationCategory.RACE); notifications.g...	A broadcast notification (no target_member_id) reaches all members.; assert len(m1_notifs) == 1; assert len(m2_notifs) == 1	PASSED (assertions satisfied)	tests/ test_notifications.py:: TestNotificationsModul test_broadcast_reaches
A targeted notification only appears for the specified member.	notifications.create_notification("Private", "Only for member 1", NotificationType.INFO, NotificationCategory.CREW, target_member_id=1)...	A targeted notification only appears for the specified member.; assert len(m1_notifs) == 1; assert len(m2_notifs) == 0	PASSED (assertions satisfied)	tests/ test_notifications.py:: TestNotificationsModul test_targeted_notificatio

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
notify_race_starting creates a RACE category HIGH priority notification.	notifications.notify_race_starting(race_id=5, race_name="Final")	notify_race_starting creates a RACE category HIGH priority notification.; assert "Final" in notif.message; assert notif.category == NotificationCategory.RACE...	PASSED (assertions satisfied)	tests/test_notifications.py::TestNotificationsModul test_convenience_race_s
notify_race_result for position 1 uses SUCCESS type and 'Victory' title.	notifications.notify_race_result(race_id=1, position=1, member_id=10, earnings=5_000.0)	notify_race_result for position 1 uses SUCCESS type and 'Victory' title.; assert notif.notification_type == NotificationType.SUCCESS; assert "Victory" in not...	PASSED (assertions satisfied)	tests/test_notifications.py::TestNotificationsModul test_convenience_race_r
notify_low_funds creates an URGENT FINANCIAL WARNING.	notifications.notify_low_funds(balance=500.0, threshold=1_000.0)	notify_low_funds creates an URGENT FINANCIAL WARNING.; assert notif.priority == NotificationPriority.URGENT; assert notif.category == NotificationCategory.FI...	PASSED (assertions satisfied)	tests/test_notifications.py::TestNotificationsModul test_convenience_low_fu
get_unread_count returns the number of unread notifications.	notifications.create_notification("N1", "msg", NotificationType.INFO, NotificationCategory.SYSTEM, target_member_id=3); notifications.c...	get_unread_count returns the number of unread notifications.; assert notifications.get_unread_count(3) == 2; assert notifications.get_unread_count(3) == 0	PASSED (assertions satisfied)	tests/test_notifications.py::TestNotificationsModul test_unread_count
Deleted notifications are no longer retrievable.	notifications.create_notification("Delete Me", "msg", NotificationType.INFO, NotificationCategory.SYSTEM); notifications.delete_notific...	Deleted notifications are no longer retrievable.; assert notifications.delete_notification(nid) is True; assert notifications.get_notification(nid) is None	PASSED (assertions satisfied)	tests/test_notifications.py::TestNotificationsModul test_delete_notification
A race can be created and retrieved by ID.	race_management.create_race("Night Race", RaceType.CIRCUIT, 5_000.0); race_management.get_race(race.race_id)	A race can be created and retrieved by ID.; assert fetched is not None; assert fetched.name == "Night Race"; assert fetched.status == RaceStatus.SCHEDULED	PASSED (assertions satisfied)	tests/test_race_management.p TestRaceManagementI test_create_race_basic
A registered driver with the DRIVER role and a good car can enter.	race_management._crew._registration.register_member("Racer", Role.DRIVER); race_management._inventory.add_car("Supra", "Toyota", 400); ra...	A registered driver with the DRIVER role and a good car can enter.; assert entry.driver.member_id == driver.member_id; assert entry.car.car_id == car.car_id	PASSED (assertions satisfied)	tests/test_race_management.p TestRaceManagementI test_enter_race_with_val

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
After entering a race, the driver is marked unavailable.	race_management._crew._registration.register_member("D", Role.DRIVER); race_management._inventory.add_car("Car", "Model", 300); race_mana...	After entering a race, the driver is marked unavailable.; assert driver.is_available is True; assert driver.is_available is False	PASSED (assertions satisfied)	tests/test_race_management.p TestRaceManagementIn test_entering_race_mark
After entering a race, the car is marked unavailable.	race_management._crew._registration.register_member("D2", Role.DRIVER); race_management._inventory.add_car("Viper", "Dodge", 600); race_m...	After entering a race, the car is marked unavailable.; assert car.is_available is False	PASSED (assertions satisfied)	tests/test_race_management.p TestRaceManagementIn test_entering_race_mark
Only DRIVER role members can enter a race.	race_management._crew._registration.register_member("Wrench", Role.MECHANIC); race_management._inventory.add_car("Car", "Sedan", 200); ra...	Only DRIVER role members can enter a race.; Expect exception via pytest.raises(InvalidEntryError)	PASSED (assertions satisfied)	tests/test_race_management.p TestRaceManagementIn test_non_driver_cannot_
A driver already in use cannot be entered again.	race_management._crew._registration.register_member("Busy", Role.DRIVER); race_management._inventory.add_car("C1", "Model", 300); race_ma...	A driver already in use cannot be entered again.; Expect exception via pytest.raises(InvalidEntryError)	PASSED (assertions satisfied)	tests/test_race_management.p TestRaceManagementIn test_unavailable_driver_
A car already in a race cannot be used in another entry.	race_management._crew._registration.register_member("D1", Role.DRIVER); race_management._crew._registration.register_member("D2", Role.DR...	A car already in a race cannot be used in another entry.; Expect exception via pytest.raises(InvalidEntryError)	PASSED (assertions satisfied)	tests/test_race_management.p TestRaceManagementIn test_same_car_cannot_b
A damaged car is rejected when entering a race.	race_management._crew._registration.register_member("D", Role.DRIVER); race_management._inventory.add_car("Clunker", "Unknown", 150); rac...	A damaged car is rejected when entering a race.; Expect exception via pytest.raises(InvalidEntryError)	PASSED (assertions satisfied)	tests/test_race_management.p TestRaceManagementIn test_damaged_car_canno
A race with no entries cannot be started.	race_management.create_race("Empty Race", RaceType.CIRCUIT, 1_000.0); race_management.start_race(race.race_id)	A race with no entries cannot be started.; Expect exception via pytest.raises(RaceManagementError)	PASSED (assertions satisfied)	tests/test_race_management.p TestRaceManagementIn test_start_race_requires_
Starting a race changes its status to IN_PROGRESS.	race_management._crew._registration.register_member("D", Role.DRIVER); race_management._inventory.add_car("Car", "Model", 300); race_mana...	Starting a race changes its status to IN_PROGRESS.; assert race.status == RaceStatus.IN_PROGRESS	PASSED (assertions satisfied)	tests/test_race_management.p TestRaceManagementIn test_start_race_changes_
Cancelling a race makes driver and car available again.	race_management._crew._registration.register_member("D", Role.DRIVER); race_management._inventory.add_car("Car", "Model", 300); race_mana...	Cancelling a race makes driver and car available again.; assert driver.is_available is False; assert driver.is_available is True; assert car.is_available is ...	PASSED (assertions satisfied)	tests/test_race_management.p TestRaceManagementIn test_cancel_race_release
For DRAG races, suitable cars are sorted by horsepower descending.	race_management._inventory.add_car("Slow", "Sedan", 150); race_management._inventory.add_car("Fast", "Sports", 600); race_management._inv...	For DRAG races, suitable cars are sorted by horsepower descending.; assert cars[0].horsepower ¿= cars[1].horsepower	PASSED (assertions satisfied)	tests/test_race_management.p TestRaceManagementIn test_get_suitable_cars_so

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Register a member with name only.	registration.register_member("Amay")	Register a member with name only.; assert member.member_id == 1; assert member.name == "Amay"; assert member.role is None	PASSED (assertions satisfied)	tests/test_registration.py::TestRegistrationModule::test_register_basic_mem
Register member with an explicit role.	registration.register_member("Vikram", Role.DRIVER)	Register member with an explicit role.; assert member.role == Role.DRIVER	PASSED (assertions satisfied)	tests/test_registration.py::TestRegistrationModule::test_register_member_w
Each new member gets a unique incrementing ID.	registration.register_member("A"); registration.register_member("B"); registration.register_member("C")	Each new member gets a unique incrementing ID.; assert m1.member_id == 1; assert m2.member_id == 2; assert m3.member_id == 3	PASSED (assertions satisfied)	tests/test_registration.py::TestRegistrationModule::test_ids_are_unique_and
Can retrieve a member by their ID.	registration.register_member("Dom", Role.DRIVER); registration.get_member(member.member_id)	Can retrieve a member by their ID.; assert fetched is not None; assert fetched.name == "Dom"	PASSED (assertions satisfied)	tests/test_registration.py::TestRegistrationModule::test_get_member_by_id
Name lookup is case-insensitive.	registration.register_member("Letty", Role.DRIVER); registration.get_member_by_name("letty"); registration.get_member_by_name("LETTY")	Name lookup is case-insensitive.; assert registration.get_member_by_name("letty") is not None; assert registration.get_member_by_name("LETTY") is not None	PASSED (assertions satisfied)	tests/test_registration.py::TestRegistrationModule::test_get_member_by_na
Fetching a nonexistent ID returns None.	registration.get_member(999)	Fetching a nonexistent ID returns None.; assert registration.get_member(999) is None	PASSED (assertions satisfied)	tests/test_registration.py::TestRegistrationModule::test_get_member_unkno
Registering with an empty name is not allowed.	registration.register_member("")	Registering with an empty name is not allowed.; Expect exception via pytest.raises(ValueError, match="empty")	PASSED (assertions satisfied)	tests/test_registration.py::TestRegistrationModule::test_empty_name_raises
Registering with only whitespace is not allowed.	registration.register_member(" ")	Registering with only whitespace is not allowed.; Expect exception via pytest.raises(ValueError)	PASSED (assertions satisfied)	tests/test_registration.py::TestRegistrationModule::test_whitespace_name_r
Count increments after each registration.	registration.register_member("X"); registration.register_member("Y"); registration.count()	Count increments after each registration.; assert registration.count() == 0; assert registration.count() == 1; assert registration.count() == 2	PASSED (assertions satisfied)	tests/test_registration.py::TestRegistrationModule::test_member_count_incr

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Removing a member decreases count and makes them unfindable.	registration.register_member("Temp"); registration.remove_member(member.member_id); registration.get_member(member.member_id)	Removing a member decreases count and makes them unfindable.; assert registration.remove_member(member.member_id) is True; assert registration.get_member(mem...	PASSED (assertions satisfied)	tests/test_registration.py::TestRegistrationModule::test_remove_member
Removing a member that doesn't exist returns False.	registration.remove_member(999)	Removing a member that doesn't exist returns False.; assert registration.remove_member(999) is False	PASSED (assertions satisfied)	tests/test_registration.py::TestRegistrationModule::test_remove_nonexistent
Two members with the same name get different IDs.	registration.register_member("Brian", Role.DRIVER); registration.register_member("Brian", Role.MECHANIC)	Two members with the same name get different IDs.; assert m1.member_id != m2.member_id	PASSED (assertions satisfied)	tests/test_registration.py::TestRegistrationModule::test_duplicate_names_ge
is_registered returns correct boolean.	registration.register_member("Check"); registration.is_registered(member.member_id); registration.is_registered(999)	is_registered returns correct boolean.; assert registration.is_registered(member.member_id) is True; assert registration.is_registered(999) is False	PASSED (assertions satisfied)	tests/test_registration.py::TestRegistrationModule::test_is_registered_check
get_all_members returns every registered member.	registration.register_member("Alpha"); registration.register_member("Beta"); registration.get_all_members()	get_all_members returns every registered member.; assert len(all_members) == 2; assert names == {"Alpha", "Beta"}	PASSED (assertions satisfied)	tests/test_registration.py::TestRegistrationModule::test_get_all_members
First place driver receives 50% of the prize pool.	self._setup_and_start_race(results, prize_pool=10_000.0); results.record_race_results(race.race_id, finishing_order=[d1.member_id, d2.me...	First place driver receives 50% of the prize pool.; assert winner_result.prize_won == 5_000.0	PASSED (assertions satisfied)	tests/test_results.py::TestResultsIntegration::test_winner_gets_50_per
Prize money is added to the inventory cash balance.	inv.get_balance(); self._setup_and_start_race(results, prize_pool=10_000.0); results.record_race_results(race.race_id, finishing_order=[...	Prize money is added to the inventory cash balance.; assert inv.get_balance() ¿ before	PASSED (assertions satisfied)	tests/test_results.py::TestResultsIntegration::test_prize_added_to_inve
Winning a race increments races_won in driver stats.	self._setup_and_start_race(results); results.record_race_results(race.race_id, finishing_order=[winner.member_id, loser.member_id]); re...	Winning a race increments races_won in driver stats.; assert stats is not None; assert stats.races_won == 1; assert stats.races_entered == 1	PASSED (assertions satisfied)	tests/test_results.py::TestResultsIntegration::test_driver_stats_record
Second place driver has races_won == 0.	self._setup_and_start_race(results); results.record_race_results(race.race_id, finishing_order=[winner.member_id, runner_up.member_id])...	Second place driver has races_won == 0.; assert stats.races_won == 0; assert stats.races_entered == 1	PASSED (assertions satisfied)	tests/test_results.py::TestResultsIntegration::test_runner_up_not_cou
Finishes in positions 1-3 are counted as podium finishes.	range(3); rm.create_race("Podium Race", RaceType.CIRCUIT, 9_000.0); rm.start_race(race.race_id)	Finishes in positions 1-3 are counted as podium finishes.; assert stats.podium_finishes == 1	PASSED (assertions satisfied)	tests/test_results.py::TestResultsIntegration::test_podium_finishes_tra

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
After recording results, drivers and cars become available again.	self._setup_and_start_race(results); results.record_race_results(race.race_id, finishing_order=[d1.member_id, d2.member_id])	After recording results, drivers and cars become available again.; assert d1.is_available is False; assert c1.is_available is False; assert d1.is_available i...	PASSED (assertions satisfied)	tests/test_results.py::TestResultsIntegration::test_resources_released_
Cars marked as damaged by damage_drivers get DAMAGED condition.	self._setup_and_start_race(results); results.record_race_results(race.race_id, finishing_order=[d1.member_id, d2.member_id], damage_driv...	Cars marked as damaged by damage_drivers get DAMAGED condition.; assert c1.condition == CarCondition.DAMAGED; assert c2.condition == CarCondition.GOOD	PASSED (assertions satisfied)	tests/test_results.py::TestResultsIntegration::test_car_damage_record
DNF drivers do not receive any prize money.	self._setup_and_start_race(results, num_drivers=2, prize_pool=5_000.0); results.record_race_results(race.race_id, finishing_order=[finis...	DNF drivers do not receive any prize money.; assert dnf_stats.dnf_count == 1; assert dnf_stats.total_earnings == 0.0	PASSED (assertions satisfied)	tests/test_results.py::TestResultsIntegration::test_dnf_driver_gets_no_
get_rankings returns drivers ordered by wins desc, earnings desc.	self._setup_and_start_race(results); results.record_race_results(race.race_id, finishing_order=[d1.member_id, d2.member_id]); results.g...	get_rankings returns drivers ordered by wins desc, earnings desc.; assert rankings[0].driver_id == d1.member_id	PASSED (assertions satisfied)	tests/test_results.py::TestResultsIntegration::test_rankings_sorted_by
Recording results for a SCHEDULED race raises ResultsError.	rm.create_race("Not Started", RaceType.DRAG, 1_000.0); results.record_race_results(race.race_id, finishing_order=[])	Recording results for a SCHEDULED race raises ResultsError.; Expect exception via pytest.raises(ResultsError)	PASSED (assertions satisfied)	tests/test_results.py::TestResultsIntegration::test_cannot_record_resu

4 Summary

The integration suite confirms module interoperability across the full StreetRace lifecycle: setup, execution, scoring, mission handling, user feedback, and ranking. Core coordination risks (resource contention, lifecycle transitions, and cross-module side effects) are explicitly tested and currently passing.